

B 3.2.2 Polymorphism Python (HL Only)



B3.2.2 Construct code to model polymorphism and its various forms, such as method overriding. (HL Only)

This section explores polymorphism in OOP, emphasising its role in code flexibility and reusability. It covers implementing dynamic polymorphism (e.g., method overriding) and applying static polymorphism to optimise code efficiency.

Introduction to Polymorphism

In object-oriented programming (OOP), polymorphism is a fundamental concept that literally means "many forms". It allows objects of different classes to respond to the same method call in their own specific way. This powerful feature contributes significantly to code flexibility and reusability, making your programs more adaptable and easier to maintain. Understanding polymorphism is crucial for tackling complex problems using OOP principles.

Polymorphism can be broadly categorised into two main types:

- **Dynamic Polymorphism (Method Overriding):** This occurs at runtime when a subclass provides a specific implementation for a method that is already defined in its superclass. The actual method that gets executed is determined by the object's type at the time of the call.
- **Static Polymorphism (Method Overloading):** While Python doesn't implement static polymorphism through method overloading in the same way as Java, its dynamic nature allows for similar flexibility through default argument values and variable argument lists.

Let's explore each of these in detail with Python examples.

Dynamic Polymorphism: Method Overriding

Method overriding is a natural part of inheritance in Python and is a key mechanism for achieving dynamic polymorphism. When a subclass defines a method with the same name as a method in its superclass, the subclass's method overrides the superclass's method.

Consider the following example involving a **Shape** superclass and its subclasses **Circle** and **Rectangle**:

```
class Shape:
    def draw(self):
        print("Drawing a shape.")

class Circle(Shape):
    def draw(self):
        print("Drawing a circle.")

class Rectangle(Shape):
    def draw(self):
        print("Drawing a rectangle.")

shapes = [Shape(), Circle(), Rectangle()]

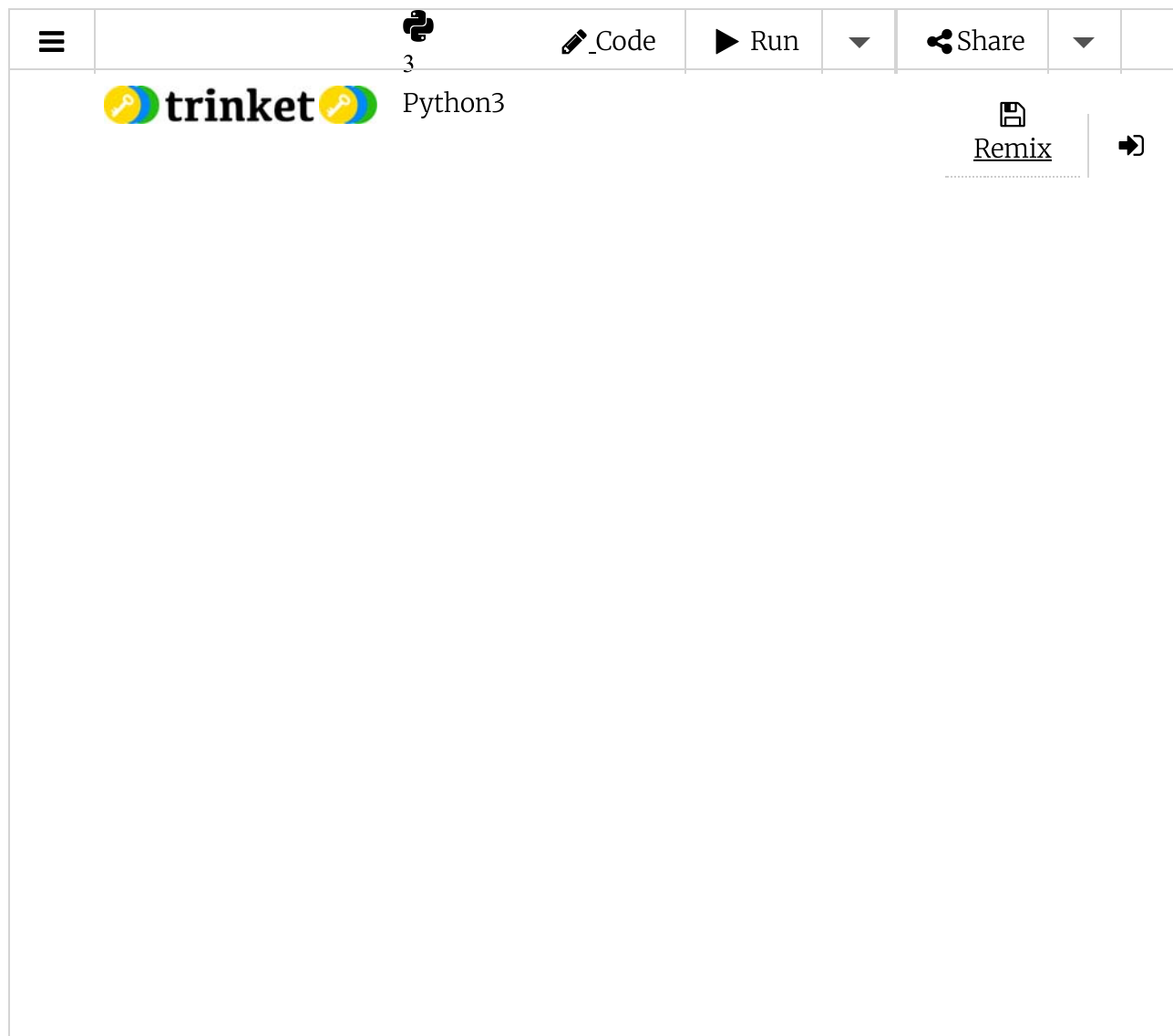
for shape in shapes:
    shape.draw()
# Output:
# Drawing a shape.
# Drawing a circle. (Dynamic Polymorphism)
# Drawing a rectangle. (Dynamic Polymorphism)
```

In this example:

- The **Shape** class has a **draw()** method.
- The **Circle** and **Rectangle** classes both inherit from **Shape** and define their own **draw()** method. This automatically overrides the **draw()** method of the **Shape** class.
- We create a list of objects, including instances of **Shape**, **Circle**, and **Rectangle**.
- When the **draw()** method is called on each object in the list, Python dynamically determines the object's type and executes the appropriate **draw()** method. This is dynamic polymorphism – the behaviour of the **draw()** method depends on the actual type of the object at runtime.

This allows for highly flexible code. You can work with different types of objects through a common method name, and each object will exhibit its specific behaviour.

Try it out



Static Polymorphism in Python

Python doesn't support traditional method overloading based on distinct parameter signatures like Java does. If you define multiple methods with the same name in a Python class, the last definition overrides earlier ones.

However, Python's dynamic typing and the use of default argument values and variable argument lists (`*args` and `**kwargs`) provide similar flexibility and allow you to create methods that can handle different numbers or types of arguments. This achieves a form of static polymorphism in practice.

Consider the following example of a `Calculator` class with a flexible `add()` method:

```
class Calculator:
    def add(self, a, b=None, c=None):
        if b is not None and c is not None:
            print("Adding three numbers.")
            return a + b + c
        elif b is not None:
            print("Adding two numbers.")
            return a + b
        else:
            print("Adding one number (itself).")
            return a

calc = Calculator()

sum1 = calc.add(5, 3)           # Calls add with two arguments - Output: Adding two
sum2 = calc.add(2.5, 3.7)       # Calls add with two arguments - Output: Adding tw
sum3 = calc.add(1, 2, 3)        # Calls add with three arguments - Output: Adding th
sum4 = calc.add(10)             # Calls add with one argument - Output: Adding one

print("Sum 1:", sum1)
print("Sum 2:", sum2)
print("Sum 3:", sum3)
print("Sum 4:", sum4)
```

In this example, the single `add()` method is designed to handle different numbers of arguments using default values. When the method is called with different numbers of arguments, the conditional logic within the method determines how to proceed. While not strict overloading in the Java sense, this approach provides a similar way to create flexible methods that can be called with varying inputs.

Try it out

Try creating some more calculations and printing the results

_CodeRunShare

3

Python3

Remix

Another way to achieve similar behaviour is by using variable arguments:

```
class FlexibleCalculator:
    def add(self, *args):
        print(f"Adding {len(args)} numbers.")
        total = 0
        for num in args:
            total += num
        return total

flex_calc = FlexibleCalculator()

sum_many1 = flex_calc.add(1, 2)           # Calls add with two arguments - Output:
sum_many2 = flex_calc.add(10, 20, 30, 40) # Calls add with four arguments - Output

print("Sum many 1:", sum_many1)
print("Sum many 2:", sum_many2)
```



Here, `*args` allows the `add()` method to accept any number of positional arguments, which are then collected into a tuple. This provides a powerful way to create methods that can work with a variable number of inputs.

Benefits of Polymorphism

Polymorphism offers several key advantages in object-oriented programming:

- **Flexibility:** It allows you to write code that can work with objects of different classes in a uniform way, making your code more adaptable to changes and extensions. Python's dynamic typing further enhances this flexibility.
- **Reusability:** You can reuse code more effectively by defining common interfaces in superclasses and allowing subclasses to provide specific implementations.
- **Maintainability:** Polymorphic code tends to be more modular and easier to understand and modify, as different behaviours are encapsulated within their respective classes.
- **Extensibility:** Adding new classes and behaviours to your program becomes easier without modifying existing code extensively. Python's "static" polymorphism approach also contributes to this by allowing flexible method definitions.

Understanding and effectively utilising polymorphism is a hallmark of good object-oriented design and a key skill in your IB Computer Science journey.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**